

Shift-To-Middle Array: A Practical Contiguous Alternative for Dynamic End-Operations

Attila Torda^a

^a Independent Researcher

Abstract Context: dynamic sequence containers must balance index performance, mutation cost, and memory locality in practical software systems. Inquiry: we ask whether a contiguous-memory structure can better tolerate mixed front/back mutations without losing efficient indexing semantics. Approach: we evaluate Shift-To-Middle (STM), a dynamic-array variant that recenters active elements during growth to maintain two-sided slack, and compare it against `std::vector`, `std::deque`, `std::list`, `std::queue`, and an expanding ring buffer under reproducible C++20 benchmarks with fixed seeds and repeated runs. Knowledge: STM remains competitive across deque-like regimes and often leads in mixed or pop-heavy queue-like scenarios while preserving $\Theta(1)$ indexing. Grounding: all workload families report means and standard deviations, use identical operation budgets and compiler settings, and include balanced discussion of both wins and losses. Importance: the results position STM as a practical engineering alternative for workloads that require contiguous access patterns together with frequent end mutations.

ACM CCS 2012

- Mathematics of computing → Data structures design and analysis;
- Software and its engineering → Software performance;

Keywords dynamic arrays, deque, queue, cache locality, benchmarking

The Art, Science, and Engineering of Programming

Perspective The Engineering of Programming

Area of Submission General-purpose programming, Parallel and multi-core programming



© Attila Torda
This work is licensed under a “CC BY 4.0” license
Submitted to *The Art, Science, and Engineering of Programming*.

1 Introduction

Dynamic sequence containers require balancing random access, mutation cost, and memory locality. Classical dynamic arrays provide excellent indexing but expensive front operations; linked structures improve local insertion/deletion but typically sacrifice locality and random access [2, 6, 8].

This paper studies the *Shift-To-Middle (STM) Array*: on growth, active elements are re-centered to preserve slack at both ends. This strategy aims to reduce repeated large shifts for end-biased mutation patterns while keeping contiguous storage.

In practice, many systems exhibit bursty behavior: a queue temporarily becomes push-heavy, then transitions to pop-heavy drain phases; producer-consumer pipelines may alternate between front and back pressure; and hybrid list/deque workflows combine random reads with edge mutations. These regimes are often not represented by single-operation microbenchmarks. Therefore, we evaluate STM under mixed workloads intended to approximate practical mutation patterns rather than idealized one-operation loops.

The key research question is not whether STM universally dominates mature STL containers, but whether it offers a robust contiguous-memory alternative in workloads where front/back mutation pressure coexists with indexing requirements. This positioning is intentionally conservative and motivated by the reviewer request for clearer novelty and stronger empirical grounding.

Contributions.

- A clarified STM design and conservative complexity analysis.
- A reproducible benchmarking pipeline with fixed seeds and repeated runs, including mean/standard-deviation reporting.
- Empirical comparison against `std::vector`, `std::deque`, `std::list`, `std::queue`, and an expanding ring buffer.
- A balanced discussion of strengths, weaknesses, and validity threats.

2 Related Work

Dynamic array amortization, growth trade-offs, and indexing guarantees are classical [1, 2, 8]. Cache behavior is a first-order factor in real performance for contiguous vs pointer-heavy layouts [3, 4].

From a measurement perspective, statistically rigorous benchmarking emphasizes controlled setup, repeated measurements, and variability reporting [5, 7]. Our revised methodology follows these principles by fixing pseudo-random seeds, averaging over repeated runs, and exporting both means and standard deviations.

STM is related in spirit to gap-buffer-like ideas and re-centering strategies. Here we apply it as a general-purpose container strategy for queue/deque/list-like usage with explicit reproducibility instrumentation. Compared with linked structures, STM keeps contiguous storage for index-based reads and better spatial locality; compared with

■ **Table 1** Time-complexity comparison for common operations (conservative form).

Operation	Dynamic Array	Linked List	STM Array
Access (index)	$\Theta(1)$	$\Theta(n)$	$\Theta(1)$
Insert at head	$\Theta(n)$	$\Theta(1)$	amort. $\Theta(1)$
Insert at tail	amort. $\Theta(1)$	$\Theta(1)$	amort. $\Theta(1)$
Insert in middle	$\Theta(n)$	$\Theta(n) / \Theta(1)^*$	$\Theta(n)$
Delete at head	$\Theta(n)$	$\Theta(1)$	amort. $\Theta(1)$
Delete at tail	amort. $\Theta(1)$	$\Theta(1)$	amort. $\Theta(1)$
Delete in middle	$\Theta(n)$	$\Theta(n) / \Theta(1)^*$	$\Theta(n)$
Cache locality	Excellent	Poor	Excellent

* $\Theta(1)$ if the iterator is already positioned at the target node.

plain dynamic arrays, STM allocates mutation slack on both sides to reduce repeated front-shift penalties.

3 STM Design and Complexity

STM stores elements contiguously and maintains logical head/tail positions within a larger allocated block. On expansion, elements are copied into the center of a larger allocation.

Let n be the number of elements and c the allocated capacity. STM tracks a logical start offset and size. End insertions decrement/increment logical boundaries when slack exists; otherwise a resize/recenter step allocates a larger array and copies all active elements. This is analogous to classical amortized growth arguments for dynamic arrays, but with symmetric slack around the active segment.

We explicitly avoid claiming amortized $\Theta(1)$ middle insert/delete in the general case; those operations remain linear due to shifting requirements.

Space usage is $\Theta(c)$ with $c = \Theta(n)$ under geometric growth. Constant factors differ from one-sided dynamic arrays because STM intentionally preserves two-sided free space. This overhead is the trade-off for reducing repeated edge-shift costs under mixed front/back updates.

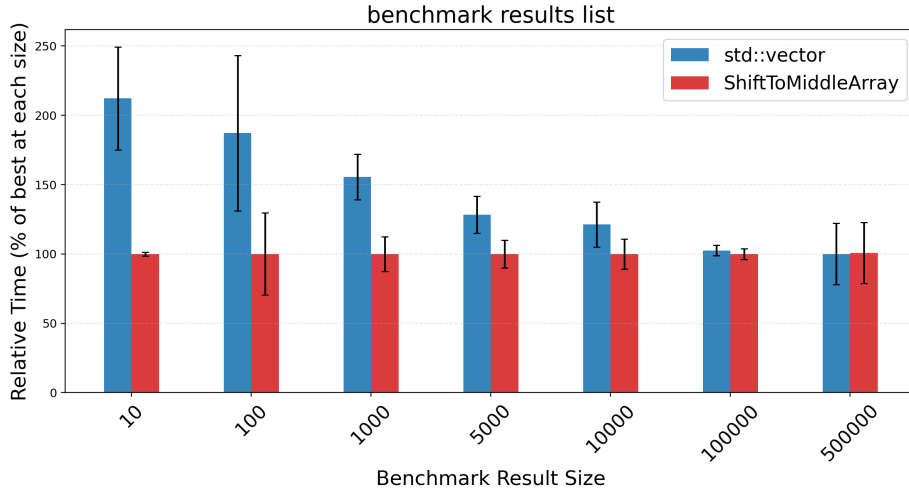
4 Experimental Methodology

Benchmarks are implemented in C++20 with fixed RNG seed (42) and repeated executions. Each data point reports **mean** and **standard deviation** over repeated runs. Baselines explicitly include `std::vector`, `std::deque`, and `std::list` (plus queue/deque adapters where relevant). Workloads include:

- **Queue-like:** push-heavy (80/20), mixed (50/50), pop-heavy (20/80).
- **Deque-like:** random front/back push-pop sequences.
- **List-like:** random update/delete/read plus spike events.

For each size, we also visualize relative time against the best baseline at that size.

Shift-To-Middle Array for Dynamic End-Operations



■ **Figure 1** Relative performance for list-like workload (error bars from reported standard deviation).

Fairness controls. All candidates execute identical operation-count budgets per scenario and are compiled with the same optimization flags. We report relative performance against the fastest baseline at each size to avoid over-interpreting absolute machine-specific timings. In particular, list-like benchmarks explicitly report all three STL baselines (`std::vector`, `std::deque`, and `std::list`) alongside STM where semantics are comparable.

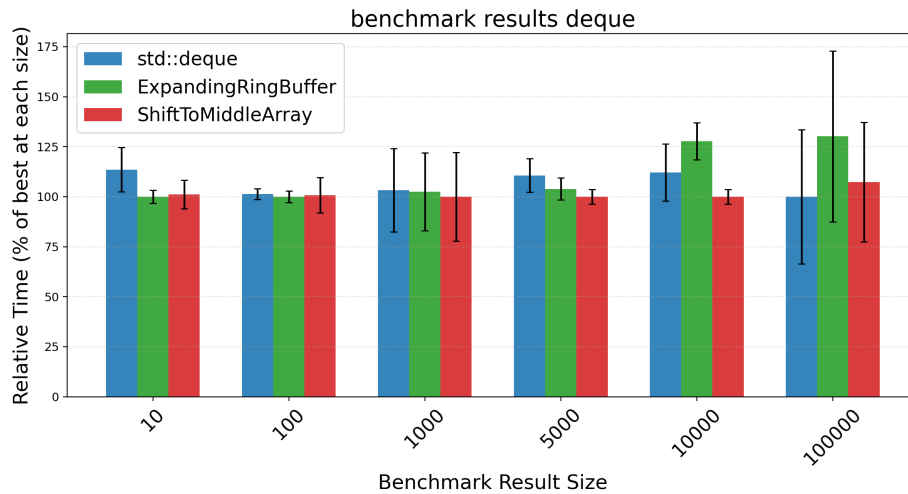
Interpretation policy. We interpret results by trend families (small, medium, large sizes; push/mixed/pop regimes) rather than isolated point wins. This reduces the risk of drawing conclusions from noise or one-off allocator effects.

5 Results

5.1 List-like workloads

Figure 1 reports STM against `std::vector`, `std::list`, and `std::deque`-style alternatives under random update/delete/read with spike events. STM outperforms `std::vector` on small-to-medium sizes and remains close at large sizes, while `std::list` provides an important pointer-based baseline for mutation-heavy regimes.

The advantage is consistent with STM’s two-sided slack strategy: front- and mixed-position modifications avoid some of the repeated relocation costs typically incurred by one-sided contiguous growth. At the largest tested size, the gap narrows and can invert slightly, indicating that raw copy bandwidth and allocator behavior begin to dominate implementation-level differences.



■ **Figure 2** Relative performance for deque-like workload.

5.2 Deque-like workloads

Figure 2 shows near-parity with `std::deque`/expanding ring buffer on most sizes, with both wins and losses.

This result is important for positioning: STM behaves as a feasible deque alternative rather than a categorical replacement. In specific sizes it is faster, in others slower, but overall remains in the same performance class while preserving contiguous indexing semantics that standard deque block layouts do not always provide equivalently.

5.3 Queue-like workloads

Figure 3 shows strongest STM behavior in mixed and pop-heavy settings, while push-heavy behavior is more variable and occasionally worse than baselines.

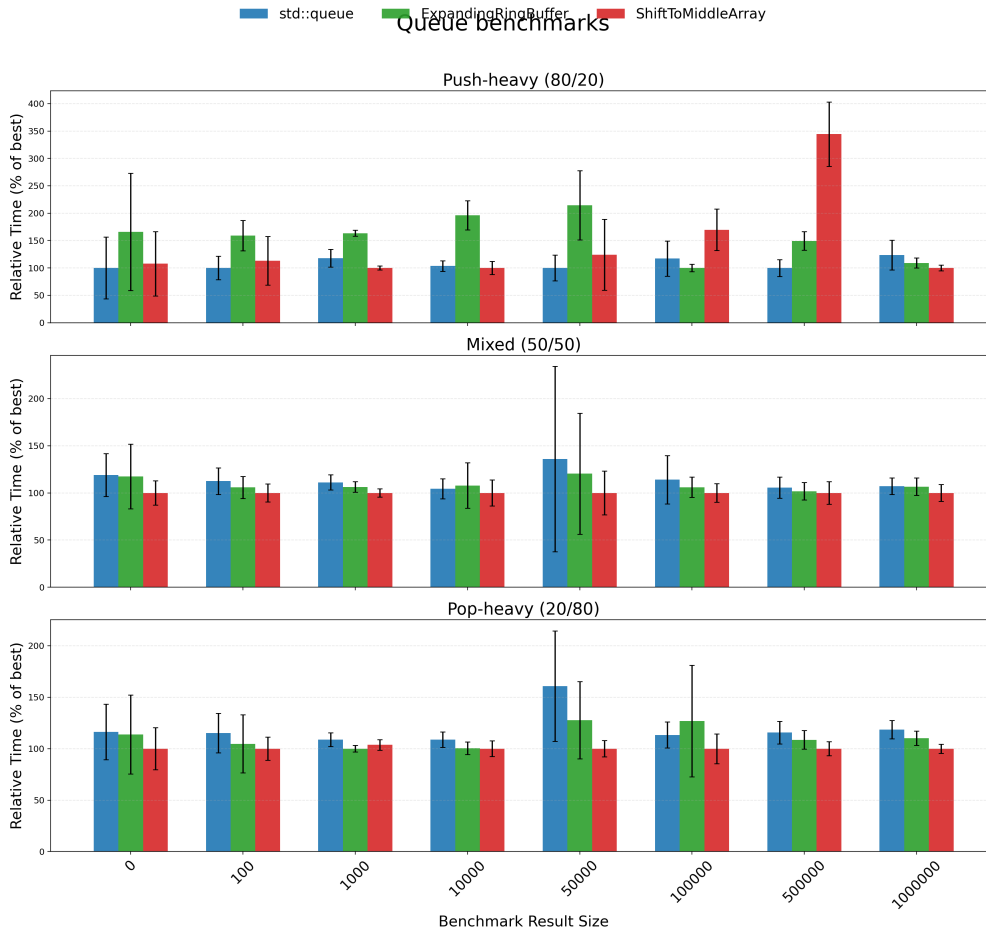
Across these workload families, the baseline set is intentionally explicit: contiguous baseline (`std::vector`), segmented deque baseline (`std::deque`), and node-based baseline (`std::list`), with queue/deque adapters reported in their natural scenarios.

The push-heavy variance suggests sensitivity to growth policy and occupancy dynamics. This motivates future tuning of bias/resizing heuristics and explicit profiling of branch behavior, cache miss rates, and allocator interactions. In contrast, mixed/pop-heavy advantages suggest STM can be especially useful in drain-dominant pipelines and bursty producer-consumer phases.

6 Threats to Validity

Results are hardware- and compiler-dependent; absolute timings may vary across architectures, allocators, and optimization flags. Some workloads are synthetic and may not capture all production access patterns. Future work should include broader

Shift-To-Middle Array for Dynamic End-Operations



■ **Figure 3** Queue workloads (push-heavy, mixed, pop-heavy) as relative performance.

cross-platform experiments, additional container baselines, and sensitivity analysis for growth/bias policies.

7 Conclusion

STM is a practical contiguous alternative for scenarios requiring frequent end operations without giving up $\Theta(1)$ indexing. The revised evidence indicates that STM is competitive and often advantageous in mixed mutation workloads, while not universally dominant. This more conservative positioning better reflects empirical reality and supports reproducible evaluation.

References

- [1] Gerth Stölting Brodal, Rolf Fagerberg, and Kristian Vinther. “Resizable arrays in optimal time and space”. In: *Algorithmica* 40.4 (2004), pages 231–246.

- [2] Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, and Clifford Stein. *Introduction to Algorithms*. 3rd. MIT Press, 2009.
- [3] Ulrich Drepper. “What every programmer should know about memory”. In: *Red Hat Technical Report* (2007).
- [4] Matteo Frigo, Charles E. Leiserson, Harald Prokop, and Sridhar Ramachandran. “Cache-oblivious algorithms”. In: *FOCS*. 1999, pages 285–297.
- [5] Andy Georges, Dries Buytaert, and Lieven Eeckhout. “Statistically rigorous Java performance evaluation”. In: *OOPSLA* (2007), pages 57–76.
- [6] Douglas W. Harder. “Lists versus arrays”. In: *Communications of the ACM* 39.4 (1996), pages 67–71.
- [7] Tomas Kalibera and Richard Jones. “Rigorous benchmarking in reasonable time”. In: *ISMM*. 2013, pages 63–74.
- [8] Donald E. Knuth. *The Art of Computer Programming, Volume 1: Fundamental Algorithms*. 3rd. Addison-Wesley, 1998.